# Reviewer Pack — Minimal Geometric Entropy of Algebraic Curves and DPLL Proof…

Entry 1e2ecb1438b2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 09:49:12 UTC

## Minimal Geometric Entropy of Algebraic Curves and DPLL Proof Path Length

**Entry ID**: `1e2ecb1438b2` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 09:49:12 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Non-Singular Curves) **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

**Statement**:

> The minimal geometric entropy of an algebraic curve associated with a CNF formula is linearly correlated with the DPLL proof path length of the formula, such that H_G(φ) = Θ(L_DPLL(φ)), where H_G(φ) is the minimal geometric entropy and L_DPLL(φ) is the DPLL proof path length.

**Rationale (proposer's reasoning)**:

> Algebraic geometry provides a structured framework to encode logical properties of CNF formulas. The geometric entropy of an algebraic curve might capture essential information about the complexity of solving the formula, providing insights into the structure of DPLL proofs that are not immediately apparent from the CNF alone.

**Taxonomy category**: `resolution_proofs` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `6f1b5b16463d9dc4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if, across at least 24 out of 30 random seeds, the correlation coefficient (r) between the minimal geometric entropy H_G(φ) and DPLL proof path length L_DPLL(φ) of CNF formulas with up to 40 variables meets r >= 0.7.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 1.00 | UNCERTAIN | SAFE |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | UNCERTAIN | 0.90 | UNCERTAIN | HITS |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `"minimal geometric entropy" AND algebraic curve` - `"DPLL proof path length" AND complexity theory` - `"CNF formula" AND H_G(φ) = θ(L_DPLL(φ))`

**Top relevant hits considered**: - [s2:e3cce2d7e072e01fa4e6518c8cef76de0fe22886] Workshop on Computational , Descriptive and Proof Complexity , and Algorithms Independent - [s2:10.1002/anie.9117281] Sequence-Defined Oligourethane Isomeric Mixtures for Irreversible Encryption.

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def dpll(cnf):
        assignment = {}

        def solve():
            unassigned_vars = [v for v in range(1, len(cnf) + 1) if v not in assignment and -v not
            if not unassigned_vars:
                return all([any([assignment.get(l, False) for l in clause]) for clause in cnf])

            p = unassigned_vars[0]
            new_assignment[p] = True
            if solve():
                return True

            del new_assignment[p]
            new_assignment[-p] = True
            if solve():
                return True
```

```
            del new_assignment[-p]
            return False

        return solve()

    n_max = 40
    instances_tested = 0
    metric_values = []

    for n in range(5, n_max + 1):
        for _ in range(6):  # Ensure at least 30 instances per seed
            cnf = generate_cnf(n)
            path_length = dpll(cnf)
            if path_length is not None:
                instances_tested += 1
                metric_values.append(path_length)

    if instances_tested < 30:
        return {
            "metric_name": "DPLL Proof Path Length",
            "metric_value": -1,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    correlation_coefficient = calculate_correlation(metric_values, [i for i in range(5, n_max + 1)

    return {
        "metric_name": "DPLL Proof Path Length",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient >= 0.7,
        "counterexample": ""
    }

def calculate_correlation(x, y):
    if len(x) != len(y):
        return None

    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n

    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    var_x = sum((x[i] - mean_x) ** 2 for i in range(n)) / n
    var_y = sum((y[i] - mean_y) ** 2 for i in range(n)) / n

    if var_x == 0 or var_y == 0:
        return None

    return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] != -1) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_s
                break
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_85f1c8f3.py", line 107, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_85f1c8f3.py", line 58, in run_trial
    path_length = dpll(cnf)
                  ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_85f1c8f3.py", line 49, in dpll
    return solve()
           ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_85f1c8f3.py", line 37, in solve
    new_assignment[p] = True
    ^^^^^^^^^^^^^^
NameError: name 'new_assignment' is not defined
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: `INCONCLUSIVE`

**Reasoning**:

> The test code crashed before producing data, which means it was unable to complete the required number of trials to evaluate the conjecture's support. | next: Review and debug the test code to ensure it can run to completion without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 12431 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 9738 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8659 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 9835 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 20677 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11125 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13884 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 12452 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 13768 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 112569 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/1e2ecb1438b2.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1e2ecb1438b2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/1e2ecb1438b2.tar.gz` (if generated)